Chapter 12

BASIC MODELING ALGORITHMS

The half-edge data structure and the Euler operators form the basic building blocks of our modeling system. In this chapter we shall start to investigate various higher-level procedures that can be built on the top of them.

12.1 MOTIVATION

According to the analogy suggested in Section 8.2.2, Euler operators and their support routines take the role of an "assembly language" in the architecture of GWB. Just as we do not (usually) want to use an assembly-level tool for expressing complicated processing, we shall need to develop various higher levels of modeling tools for geometric modeling.

One of the major virtues of boundary representations is the opportunity to use various sorts of "local modifications" that can generate frequently needed solids without the large computational burden of, say, Boolean set operations. This chapter takes a brief look into a family of such "medium-level" operations useful for implementing parameterized primitives and some drawing-type operations directly on top of Euler operators.

As usual, we aim at a concise and economical implementation. As will be seen, this leads to a situation where the various procedures rely on each other in a fashion that makes it appropriate to consider these routines an independent layer of its own right in the architecture of GWB.

12.2 ARC GENERATOR

A typical case for a higher-level operation that should be implemented on the top of Euler operators is the generation a polygonal approximation of a circular arc. Obviously, all arc generation methods discussed in Chapter 2 are of potential interest, and a serious modeler would implement most of them and probably others.

Here we shall discuss only the simplest case, and develop a procedure that can generate an arc based on the radius and the coordinates of the center. Even this is subject to the restriction that the plane of definition is of the form $z = h$. Furthermore, we assume that the first vertex of the arc already exists, and supply its identifier to the procedure.

This specification is implemented in Program 12.1. The procedure **arc** generates an approximation of a circular arc segment with n edges, centered at $(cx\ cy)$, on plane $z = h$, and with radius *rad*. The arc ranges from angle *phi1* to *phi2*, measured in degrees, where $0.0 = x$-axis and angles grow counterclockwise. The arc starts from existing vertex v of face f.

The algorithm must be capable of producing unique new vertex names. Simplistically, this is implemented by means of a procedure **getmaxnames** that stores the largest vertex and face identifiers of its argument solid in global variables *maxv* and *maxf*, respectively.

As given, the procedure is not capable of producing a full circle, or closing a polygon. A general solution must check whether the last point of the arc actually is a vertex of f, and if so, apply MEF instead of MEV to generate the last edge of the arc. To just generate a circle, a simpler alternative is to build a circle generator on the top of **arc** as in Program 12.2.

12.3 SWEEPING PRIMITIVES

Translational and rotational sweeping are other kinds of convenient solid description techniques that can be implemented in terms of Euler operators.

12.3.1 Translational Sweeping

Let us first consider the *face sweep* operation that takes a face f of a solid, and sweeps it along a vector $[dx\ dy\ dz]$. Note that the profile sweep operation is a special case of this, because a closed profile is represented as a *lamina* having two faces like the two sides of a piece of paper. Hence profile sweep amounts to sweeping one or the other of the faces of the lamina.

In this algorithm low-level Euler operators are the most natural tools. The algorithm of Program 12.3 traverses each loop of the argument face,


```
void    arc(s, f, v, cx, cy, rad, h, phi1, phi2, n)
Id      s, f, v;
float   cx, cy, rad, h, phi1, phi2;
int     n;
{
        float   x, y, angle, inc;
        Id      prev;
        int     i;

        angle = phi1 * PI / 180.0;
        inc = (phi2 - phi1) * PI / (180.0 * n);
        prev = v;
        getmaxnames(s);
        for(i = 0; i < n; i++)
        {
                angle += inc;
                x = cx + cos(angle) * rad;
                y = cy + sin(angle) * rad;
                smev(s, f, prev, ++maxv, x, y, h);
                prev = maxv;
        }
}

void    getmaxnames(sn)
Id      sn;
{
        Solid   *s;
        Vertex  *v;
        Face    *f;

        s = getsolid(sn);
        for(v = s->sverts, maxv = 0; v != NIL; v = v->nextv)
                if(v->vertexno > maxv) maxv = v->vertexno;
        for(f = s->sfaces, maxf = 0; f != NIL; f = f->nextf)
                if(f->faceno > maxf) maxf = f->faceno;
}
```

Program 12.1 Arc generator.

```
Solid   *circle(sn, cx, cy, rad, h, n)
Id      sn;
float   cx, cy, rad, h;
int     n;
{
        Solid   *s;

        s = mvfs(sn, 1, 1, cx+rad, cy, h);
        arc(sn, 1, 1, cx, cy, rad, h, 0.0, ((n-1)*360.0/n), n-1);
        smef(sn, 1, n, 1, 2);
        return(s);
}
```

Program 12.2 Circle generator.

and adds a new edge from each vertex along the sweep direction with `lmev`, the low-level counterpart of MEV. The end vertices of these new edges are joined with `lmef` to create side faces of the swept region. The algorithm is implemented so that the final top face of the swept region will have the identical identifier as the original face.

As an example of the use of `sweep`, the rectilinear box and cylinder primitives can be implemented as in Program 12.4. Because `sweep` needs a pointer to the argument face, the procedure `fface` described in Chapter 11 is used.

12.3.2 Rotational Sweeping

Rotational sweeping "swings" a two-dimensional figure around an axis to create a rotational solid. Of course, we shall create a polyhedral approximation of the rotational surface only; the accuracy of the approximation is selected by parameter *nfac* that tells how many points are used to approximate any circle of the solid.

To keep things simple, let us first consider the most straightforward case where the rotation axis is the x-axis, and the figure is a simple, connected, and nonforking string of edges, all lying on the half-plane $y > 0, z = 0$. Such an object is called a *wire*.

The algorithm we shall develop performs the sweeping operation in two phases. Referring to Figure 12.1(a–c), in the first phase the algorithm makes $nfaces - 1$ scans along the wire, and creates one new face for each original wire edge with an algorithm somewhat similar to the translational sweeping algorithm. After the final scan, we have the arrangement of Fig-


```
void    sweep(fac, dx, dy, dz)
Face    *fac;
float   dx, dy, dz;
{
        Loop            *l;
        HalfEdge        *first, *scan;
        Vertex          *v;

        getmaxnames(fac->fsolid);
        l = fac->floops;
        while(l)
        {
                first = l->ledg;
                scan = first->nxt;
                v = scan->vtx;
                lmev(scan, scan, ++maxv,
                        v->vcoord[0] + dx,
                        v->vcoord[1] + dy,
                        v->vcoord[2] + dz);
                while(scan != first)
                {
                        v = scan->nxt->vtx;
                        lmev(scan->nxt, scan->nxt, ++maxv,
                                v->vcoord[0] + dx,
                                v->vcoord[1] + dy,
                                v->vcoord[2] + dz);
                        lmef(scan->prv, scan->nxt->nxt, ++maxf);
                        scan = mate(scan->nxt)->nxt;
                }
                lmef(scan->prv, scan->nxt->nxt, ++maxf);
                l = l->nextl;
        }
}
```

Program 12.3 Translational sweeping.

```
Solid   *block(sn, dx, dy, dz)
Id      sn;
float   dx, dy, dz;
{
        Solid       *s;

        s = mvfs(sn, 1, 1, 0.0, 0.0, 0.0);
        smev(sm, 1, 1, 2, dx, 0.0, 0.0);
        smev(sn, 1, 2, 3, dx, dy, 0.0);
        smev(sn, 1, 3, 4, 0.0, dy, 0.0);
        smef(sn, 1, 1, 4, 2);
        sweep(fface(s, 2), 0.0, 0.0, dz);
        return(s);
}

Solid   *cyl(sn, rad, h, n)
Id      sn;
float   rad, h;
int     n;
{
        Solid       *s;

        s = circle(sn, 0.0, 0.0, rad, 0.0, n);
        sweep(fface(s, 2), 0.0, 0.0, h);
        return(s);
}
```

Program 12.4 Block and cylinder primitives.

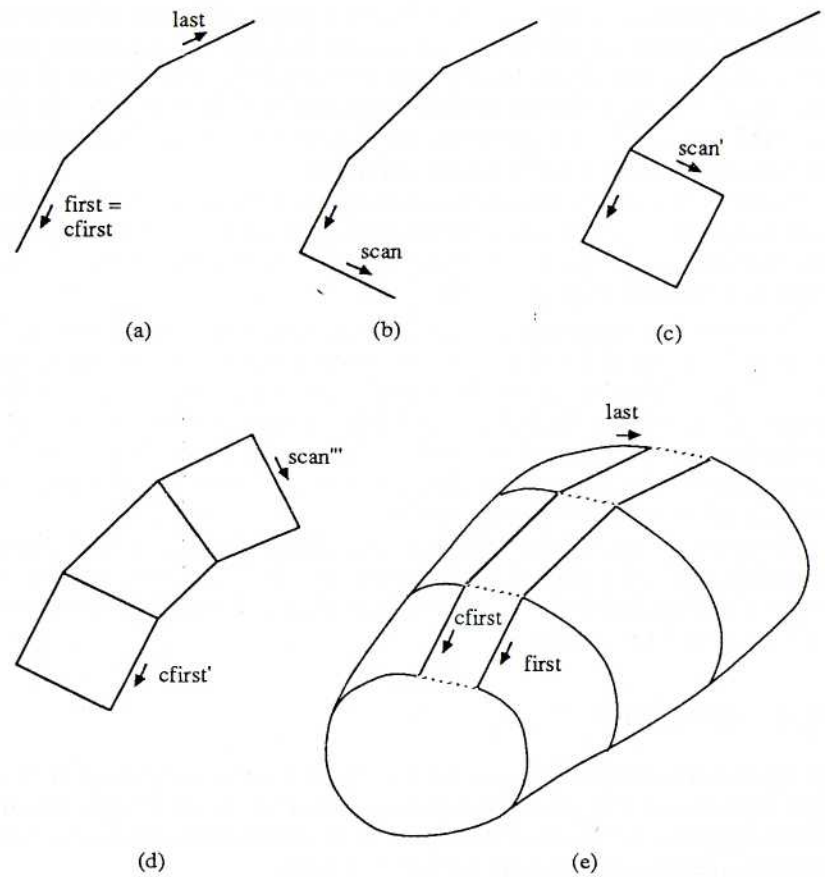

Figure 12.1 Operation of the rotational sweeping algorithm.

ure 12.1(d), where the most recent edges created must still be joined with the edges of the original wire. This is implemented in the second phase.

The procedure of Program 12.5 implements the algorithm by first finding the head *first* of the wire; variables *cfirst* and *clast* are used to hold the leftmost edge of the current scan (see Figure 12.1(a)). The first while-loop controls the *nfaces* − 1 wire scans, while inner loop marches along the wire and creates the new edges; *clast* is used to control the termination of the inner loop. This leads us the situation depicted in Figure 12.2(d). In the last "mending" scan, the remaining faces and edges are created. Again, the best approach is to use low-level operators.

Based on the rotational sweeping algorithm of Program 12.5, it would seem tempting to implement a ball primitive as shown in Program 12.6. The procedure ball displayed works by generating a half-circle of *nver* edges, and rotating it in *nhor* steps.

Unfortunately, while the graphical result may seem correct, the algorithm of Program 12.5 will rotate also the end vertices of the arc (lying on the x-axis), generating two faces with *nhor* coincident vertices at the "poles" of the sphere. Such faces are likely to cause problems in numerical applications, and should be avoided. The proper generalization of the rotational sweeping algorithm to deal with the case when the wire hits the x-axis is left as an exercise to the reader.

Observe also that the algorithm is not capable of rotating a closed figure; i.e., solids with holes cannot be produced with it. The generalization of Program 12.5 to do this will be tackled in Section 12.5 after some necessary machinery has been developed.

12.4 GLUING

All algorithms of this chapter so far have been *local modifications* in that they work on one half-edge data structure instance. Another approach for solid description is the use of various sorts of *combination operations* that combine simpler solids to more complicated ones.

Prime examples of these operations are, of course, the Boolean set operations. As they (and related advanced operations) are the topic of Chapters 14 and 15, we shall here take a look into the much simpler problem of "gluing" two solids over a common face.

Let us consider the arrangement of two 6-sided cylinders, positioned right next to each other as depicted on the left side of Figure 12.2. Our aim is to glue the two cylinders into one taller solid like the one on the right side. (Observe that the cylinders meet along a pair of entirely coincident faces.)


```
void    rsweep(s, nfaces)
Solid   *s;
int     nfaces;
{
        HalfEdge        *first, *cfirst, *last, *scan;
        Face            *tailf;
        matrix          m;
        vector          v;

        getmaxnames(s);
        first = s->sfaces->floops->ledg;
        while(first->edg != first->nxt->edg) first = first->nxt;
        last = first->nxt;
        while(last->edg != last->nxt->edg) last = last->nxt;
        cfirst = first;
        matident(m);
        matrotate(m, (360.0 / nfaces), 0.0, 0.0);
        while(--nfaces)
        {
                vecmult(v, cfirst->nxt->vtx->vcoord, m);
                lmev(cfirst->nxt, cfirst->nxt, ++maxv,
                        v[0], v[1], v[2]);
                scan = cfirst->nxt;
                while(scan != last->nxt)
                {
                        vecmult(v, scan->prv->vtx->vcoord, m);
                        lmev(scan->prv, scan->prv, ++maxv,
                                v[0], v[1], v[2]);
                        lmef(scan->prv->prv, scan->nxt, ++maxf);
                        scan = mate(scan->nxt->nxt);
                }
                last = scan;
                cfirst = mate(cfirst->nxt->nxt);
        }
        tailf = lmef(cfirst->nxt, mate(first), ++maxf);
        while(cfirst != scan)
        {
                lmef(cfirst, cfirst->nxt->nxt->nxt, ++maxf);
                cfirst = mate(cfirst->prv)->prv;
        }
}
```

Program 12.5 Rotational sweeping.

```
Solid   *ball(r, nver, nhor)
float   r;
int     nver, nhor;
{
        Solid           *s;

        s = mvfs(1, 1, -r, 0.0, 0.0);
        arc(s, f, 1, 0.0, 0.0, r, 0.0, 180.0, 0.0, nver);
        rsweep(s, nhor);
        return(s);
}
```

Program 12.6 Ball primitive.

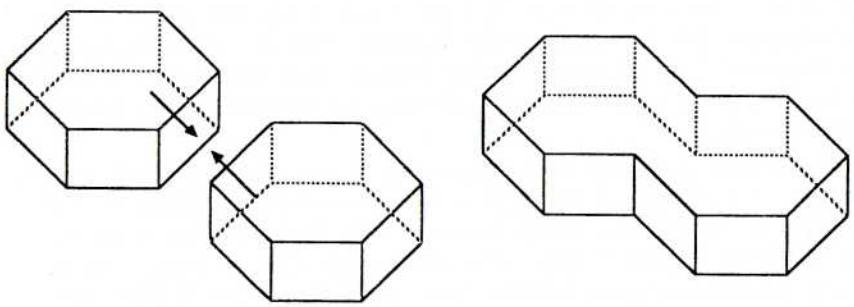

Figure 12.2 Gluing of two cylinders.


```
void    glue(s1, s2, f1, f2)
Solid   *s1, *s2;
Face    *f1, *f2;
{
        merge(s1, s2);
        lkfmrh(f1, f2);
        loopglue(f1);
}
```

Program 12.7 The gluing procedure.

The solution of the gluing problem splits into three parts, the combination of which gives the desired procedure of Program 12.7:

1. The merging of two half-edge data structures into one data structure that has two shells.
2. The joining of the shells. This amounts to applying a `lkfmrh` to the two glued faces.
3. The merging of coincident edges and vertices of the face resulting from `lkfmrh`.

12.4.1 Joining of Solids

The first step of the gluing algorithm, the procedure `merge`, joins two solids by making their all faces, edges, and vertices appear in one half-edge data structure instance. Using the procedures `dellist` and `addlist` described in Chapter 11, it is readily implemented as in Program 12.8.

12.4.2 Loop Gluing

After `merge`, the two glue faces can be joined with a `lkfmrh`. Here `lkfmrh` really acts as a "shell deletion" operator: it modifies the object by reducing a disconnected object to a connected one. The result of this is a face consisting of two loops, both with the identical number of vertices going pairwise through identical points. This arrangement is pictured by the plane model on the left side of Figure 12.3.

While the face satisfies the topological integrity criteria, we do not want to have such things in a "finished" model. Clearly, the desired arrangement in this case is the arrangement of the right side, where "identical" vertices

```
void    merge(s1, s2)
Solid   *s1, *s2;
{
        while(s2->sfaces)
        {
                dellist(FACE, s2->sfaces, s2);
                addlist(FACE, s2->fsaces, s1);
        }
        while(s2->sedges)
        {
                dellist(EDGE, s2->sedges, s2);
                addlist(EDGE, s2->sedges, s1);
        }
        while(s2->sverts)
        {
                dellist(VERTEX, s2->sverts, s2);
                addlist(VERTEX, s2->sverts, s1);
        }
        del(SOLID, s2, NIL);
}
```

Program 12.8 Joining of solids.

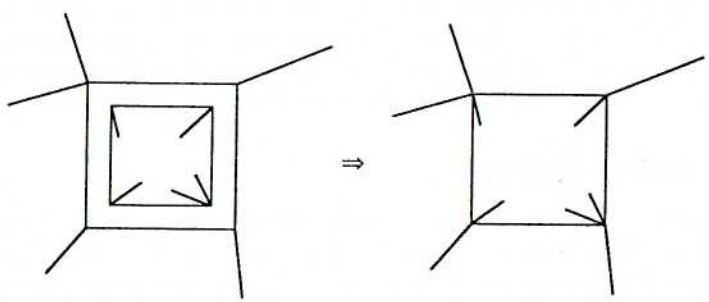

Figure 12.3 Loop gluing.

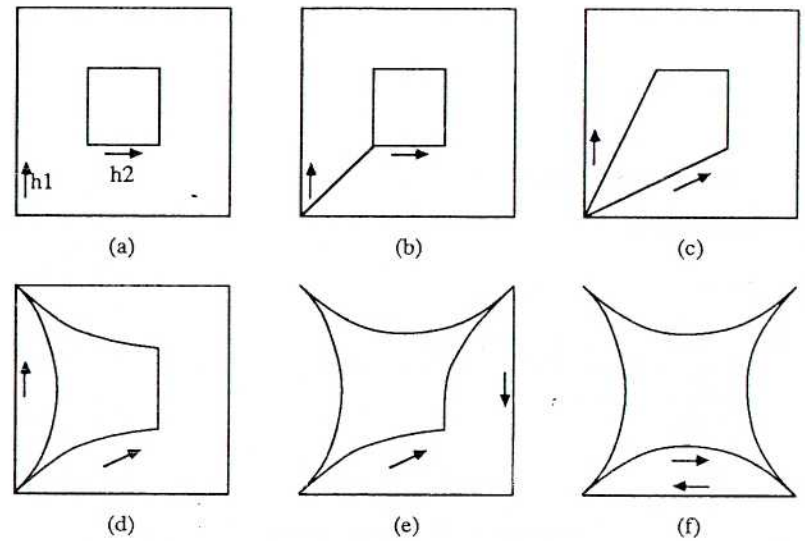

Figure 12.4 Operation of the loop gluing algorithm.

and edges have pairwise been combined, and the "glue" face has completely disappeared.

The transformation of Figure 12.3 is the task of the *loop gluing* procedure `loopglue`. It is best implemented in terms of low-level Euler operators as in Program 12.9. The algorithm first locates a pair of halfedges *h1* and *h2* whose vertices "match," and then advances *h1* around the loop, joining edges and vertices.

The detailed operation of the algorithm is best explained with help of Figure 12.4. The first `lmekr` joins the two matching vertices with a new edge, which is immediately removed with `lkev`, leading us to the arrangement of Figure 12.4(c). The `while` loop adds an edge between the next matching vertex pair with a `lmef`, and joins them with `lkev`. One of the identical edges is removed. Observe that the temporary face with identifier -1 will be immediately removed by the `lkef` that follows. The loop terminates with the arrangement of Figure 12.4(f), and the last `lkef` removes the last duplicate edge.

This algorithm joins all matching pairs of vertices with new edges, and collapses the vertices together by removing the edge immediately. As these intermediate edges join two vertices with identical coordinates, they do

```
void    loopglue(fac)
Face    *fac;
{
        HalfEdge        *h1, *h2, *h1next;

        h1 = fac->floops->ledg;
        h2 = fac->floops->nextl->ledg;
        while(!match(h1, h2)) h2 = h2->nxt;
        lmekr(h1, h2);
        lkev(h1->prv, h2->prv);
        while(h1->nxt != h2)
        {
                h1next = h1->nxt;
                lmef(h1->nxt, h1->prv, -1);
                lkev(h1->nxt, mate(h1->nxt));
                lkef(mate(h1), h1);
                h1 = h1next;
        }
        lkef(mate(h1), h1);
}

int             match(h1, h2)
HalfEdge        *h1, *h2;
{
        ...
}
```

Program 12.9 The loop gluing procedure.


```
Solid   *torus(sn, r1, r2, nf1, nf2)
Id      sn;
float   r1, r2;
int     nf1, nf2;
{
        Solid       *s;

        s = circle(sn, 0.0, r1, r2, 0.0, nf2);
        rsweep(s, nf1);
        return(s);
}
```

Program 12.10 Torus primitive.

not quite satisfy our notion of a geometrically sensible edge. Later it will turn out, however, that various arrangements involving these *null edges* are useful when manipulating half-edge data structures with Euler operators.

12.5 ROTATIONAL SWEEPING REVISITED

While developed above for a rather peculiar use, `loopglue` will have many uses. The reader may be surprised to see that it is vital for generalizing the rotational sweeping algorithm as follows.

Recall that the rotational sweeping algorithm was designed so as to operate on an *open* polygonal string of edges, a wire. Hence without modifying the rotational sweeping algorithm of Program 12.5 we cannot generate a torus by the algorithm of Program 12.10 because the circle is a closed figure, i.e., a lamina with two faces.

The solution is surprisingly simple. First, we "open" the lamina at an arbitrary vertex by duplicating it with a MEV, and removing the null edge connecting the resulting two vertices with a KEF. This gives us an open wire (albeit it has a pair of vertices lying at the same point) that can be swung with the procedure `rsweep`.

Consider now the arrangement that arises after these steps (see Figure 12.5(c)). The two copies of the split vertex cause two circular faces, right on top of each other, to appear in the result of `rsweep`. This obviously is a "canonical" case where KFMRH will do the right thing by subtracting one of the faces from the other and thereby creating a hole. After that, we have a face with two exactly matching loops—i.e., exactly the arrangement where `loopglue` produces the desired answer.

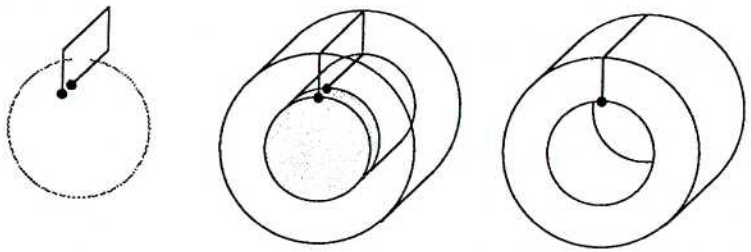

Figure 12.5 Swinging a closed figure.

The changes needed in the procedure **rsweep** amount to inserting the code segments of Program 12.11 into the code of Program 12.5, before and after all other processing.

That is, if the solid to be swept is not a wire, it is assumed to be a lamina. The lamina is broken at an arbitrary vertex to yield a single face *headf*. After the ordinary rotational sweep, the "end" faces *headf* and *tailf* of the resulting object are merged together. (This generalization was anticipated in Program 12.5 by already setting the variable *tailf* only needed here.)

PROBLEMS

12.1. Write a procedure for creating a *fillet arc* at a sharp corner. The procedure can have the following calling sequence:

```
int         fillet(he, radius)
HalfEdge    *he;
float       radius;
```

That is, the procedure is expected to replace the vertex **he->vtx** with an arc of the specified radius so that the arc connects tangentially with the neighboring edges (i.e., **he->prv->edg** and **he->edg**).

Hint: Calculate first the center and the end points of the arc. Subdivide the neighboring edges at the end points, and use a modification of the procedure **arc** of Program 12.1 to actually create the fillet arc. Thereafter, remove the unnecessary remainders of the original edges. Special cases of the operation are

(a) the angle formed by the edges is zero or 180 degrees;

(b) the radius is too large, i.e., the end points do not occur within the neighbor edges; and


```
void    rsweep(s, nfaces)
Solid   *s;
int     nfaces;
{
        ...
        HalfEdge        *h;
        Face            *headf;
        int             closed_figure;

        if(s->sfaces->nextf)  /* does the solid have > 1 faces ? */
        {
                /* assume it's a lamina */
                        closed_figure = 1;
                h = s->sfaces->floops->ledg;
                lmev(h, mate(h)->nxt, ++maxv,
                        h->vtx->vcoord[0],
                        h->vtx->vcoord[1],
                        h->vtx->vcoord[2]);
                lkef(h->prv, mate(h->prv));
                        headf = s->sfaces;
        }
        else    closed_figure = 0;              /* no, it's a wire */
        ...
        if(closed_figure == 1)
        {
                lkfmrh(headf, tailf);
                loopglue(headf);
        }
}
```

Program 12.11 Rotational sweeping of closed figures.

(c) the fillet arc "eats up" one or both of the neighbor edges, i.e., the fillet radius is the largest permitted by item (b) above.

12.2. Generalize the rotational sweeping algorithm of Program 12.5 to work correctly in the case that one or both end points of the swept wire lie on the x-axis.

Hint: Write an algorithm that reduces a face to a single vertex, and apply it to the faces *headf* and *tailf* of Program 12.11.

12.3. Generalize the modified rotational sweeping algorithm of Program 12.11 to work correctly with closed figures that have an edge lying on x-axis.

12.4. Write the procedure

```
int     unsweep(f)
Face    *f;
```

that will "flatten" a solid generated by sweeping the face f. (Observe that f will appear as the "top" face of the swept part.)

BIBLIOGRAPHIC NOTES

The story of implementing various higher-level tools on the top of Euler operators goes on. Solid modelers BUILD-2 [18] and ROMULUS include a wide collection of "local manipulations" (various forms of sweeping, tweaking, bending, etc.) implemented on top of Euler operators. MODIF [25] allows the user to employ Euler operators in quite a direct fashion to define a polyhedral "initial" solid as the initial model for generating curved surfaces. In similar vein, also ROMULUS supports "drafting"-type operations that give the user considerable freedom as for creating geometric objects on top of Euler operators.